

Design and Evaluation of Static, Dynamic, and MPI-Based Distributed Bitcoin Mining Systems

Bolla Lokesh Reddy - CS23BTECH11011
Muvvala Sairam Suhas - CS23BTECH11038

24th November 2025

Contents

1	Introduction	2
2	Transaction Input and SHA-256 Mining	2
3	Miners	2
3.1	Static Partition Mining	2
3.2	Dynamic Work-Stealing Mining	3
3.3	MPI-Based Distributed Mining	3
4	Experimental Results	5
4.1	Static Partition Miner Results	5
4.1.1	Threads vs Time	5
4.1.2	Difficulty vs Time	6
4.2	Dynamic Work-Stealing Miner Results	7
4.2.1	Threads vs Time	7
4.2.2	Difficulty vs Time	8
4.3	MPI Distributed Miner Results	9
4.3.1	Processes vs Time (Difficulty 5)	10
5	Overall Comparison	12
6	Final Observations and Insights	12
7	Conclusion	14

1 Introduction

This report presents the design and implementation of a Bitcoin mining framework that supports three different execution models: static multithreaded partitioning, dynamic work-stealing parallelism, and fully distributed mining using MPI with Raft-based leader election. The system uses SHA-256 hashing to search for valid nonces that satisfy a given difficulty target.

Bitcoin mining requires repeatedly computing SHA-256 hashes of a transaction plus a nonce until a hash with a required number of leading zeros (the *difficulty*) is found. To accelerate this search, mining is typically parallelized. In this project we implemented:

- A **static partition miner**
- A **dynamic work-stealing miner**
- A **distributed MPI miner** with leader election

2 Transaction Input and SHA-256 Mining

Transactions are read from `transactions.txt`. Mining computes:

$$hash = SHA256(transaction_data || nonce)$$

3 Miners

3.1 Static Partition Mining

Implementation Details

The static miner is implemented in `static_partition.cpp`. Each thread receives a fixed, non-overlapping nonce interval. The program:

- Reads transaction data from `transactions.txt`
- Splits the search space into equal partitions
- Launches one thread per partition
- Computes SHA-256 as `data + nonce`, matching the MPI miner format
- Stops all threads once any thread finds a valid hash
- Logs progress into `logs_static/`

The key structures include:

- `MinerArgs`: holds per-thread nonce range and configuration
- `global_found`, `global_nonce`: shared atomic signals

This method is simple but can suffer from imbalance if the winning nonce lies near the start of one partition.

3.2 Dynamic Work-Stealing Mining

Implementation Details

The dynamic miner is implemented in `dynamic_stealing.cpp`. Unlike the static version, work is divided into small chunks which threads can steal from each other. The miner:

- Loads transaction data from `transactions.txt`
- Divides the nonce space into chunks of configurable `chunk.size`
- Assigns these chunks round-robin to workers
- Allows threads with empty queues to steal:
 - From the back of another worker’s queue
 - Or split a victim’s chunk if only one remains
- Computes SHA-256 using the same format as MPI miner
- Stops all workers when a valid nonce is found

Implementation uses:

- `WorkerState`: each worker’s deque of nonce ranges
- Atomic counters: `global_found`, `total_hashes`
- Randomized victim selection to reduce lock contention

Dynamic load balancing allows better CPU utilization and prevents idle threads, particularly at higher difficulties.

3.3 MPI-Based Distributed Mining

Implementation Details

The distributed miner is implemented in `mpi_mining.cpp`, which coordinates multiple processes across machines using MPI and implements a custom Raft-like consensus layer for leader election, work management, and fault tolerance. The host configuration for launching MPI processes is stored in `hosts.txt`, and all mining tasks operate on transaction data loaded from `transactions.txt`.

Architecture Overview The distributed system consists of:

- Multiple MPI processes (possibly across different VMs or CPU cores)
- A Raft-based leader responsible for assigning nonce ranges
- Followers that perform mining and report progress/results

Key Components

- **RaftState:** Stores state information such as term, leader ID, vote information, and heartbeat timers.
- **WorkRange / WorkAssignment:** Represents a chunk of nonce space assigned to a worker.
- **WorkStatusReport:** Periodic checkpoint messages from workers to the leader.
- **WorkResult:** Returned by workers when a mining range finishes (either solution found or exhausted).

Workflow

1. **Transaction Broadcast:** Rank 0 reads all transactions from the file and broadcasts them to every MPI rank using `MPI_Bcast`. A probabilistic selection mechanism ensures diverse transactions are occasionally chosen.
2. **Leader Election (Raft):** When a leader is not present (e.g., on startup or timeout), nodes begin a Raft election:
 - Request votes using `TAG_REQUEST_VOTE`
 - Receive replies using `TAG_VOTE_REPLY`
 - A node becomes leader after majority approval
3. **Work Distribution:** The leader splits the global nonce search space into *WorkRange* elements and assigns them to workers with `MPI_Send`. Work is expanded dynamically when space runs out.
4. **Mining on Workers:** Each follower:
 - Receives a work assignment with transaction data and nonce range
 - Computes SHA-256 on `data + nonce`
 - Sends progress checkpoints periodically
 - Reports a found solution immediately to the leader
5. **Leader Monitoring:** The leader continuously:
 - Sends heartbeats (to prevent elections)
 - Receives hash progress / checkpoint updates
 - Issues new work when a range finishes
 - Terminates all nodes if a valid nonce is found
6. **Solution Broadcast:** Once any worker finds a valid nonce:
 - Leader writes the result to `final_result.txt`
 - Notifies all processes with `TAG_TERMINATE`
 - A final global solution broadcast synchronizes results

Fault Tolerance Through periodic checkpoints and consistent Raft state:

- Work ranges can be reassigned if a worker fails
- Elections guarantee progress if the leader stops responding
- Global work state is saved and restored as needed

Summary Compared to static and dynamic miners, the MPI miner:

- Distributes work across physical machines
- Dynamically expands the global nonce search range
- Uses consensus to maintain correctness under failures
- Handles message passing, heartbeats, checkpointing, and termination

This makes it the most fully featured and scalable mining framework implemented in the project.

4 Experimental Results

4.1 Static Partition Miner Results

4.1.1 Threads vs Time

Threads	Time (ms)
2	1480
4	1250
8	1096
12	410
16	141

Table 1: Static Miner: Time vs Threads (difficulty: 5, total-nonces: 50000000)

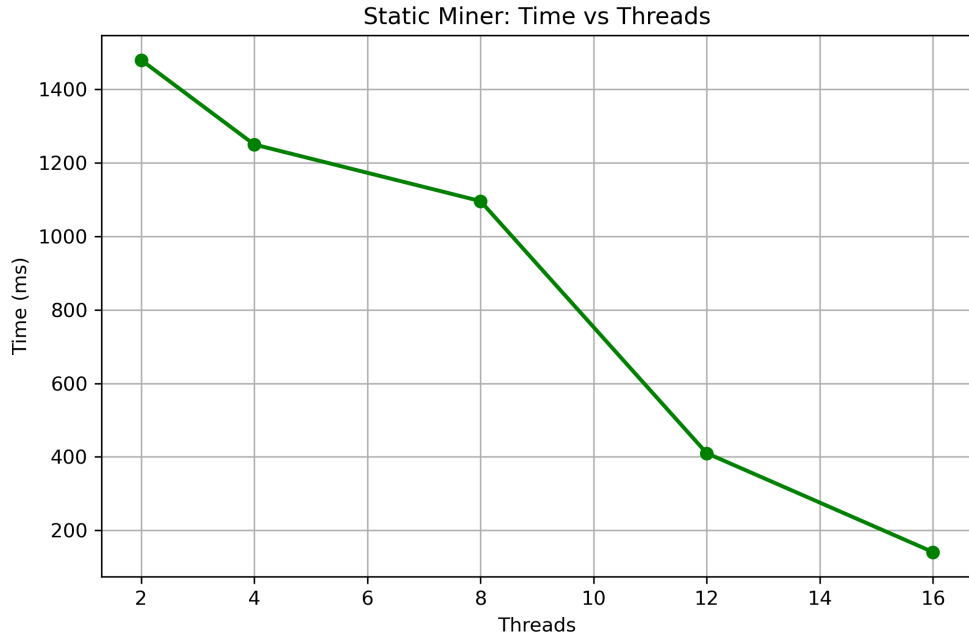


Figure 1: Static Miner: Time vs Threads

4.1.2 Difficulty vs Time

Difficulty	Time (ms)
2	3
3	27
4	144
5	613
6	2499

Table 2: Static Miner: Time vs Difficulty (threads fixed, Threads: 6 total-nonces: 50000000)

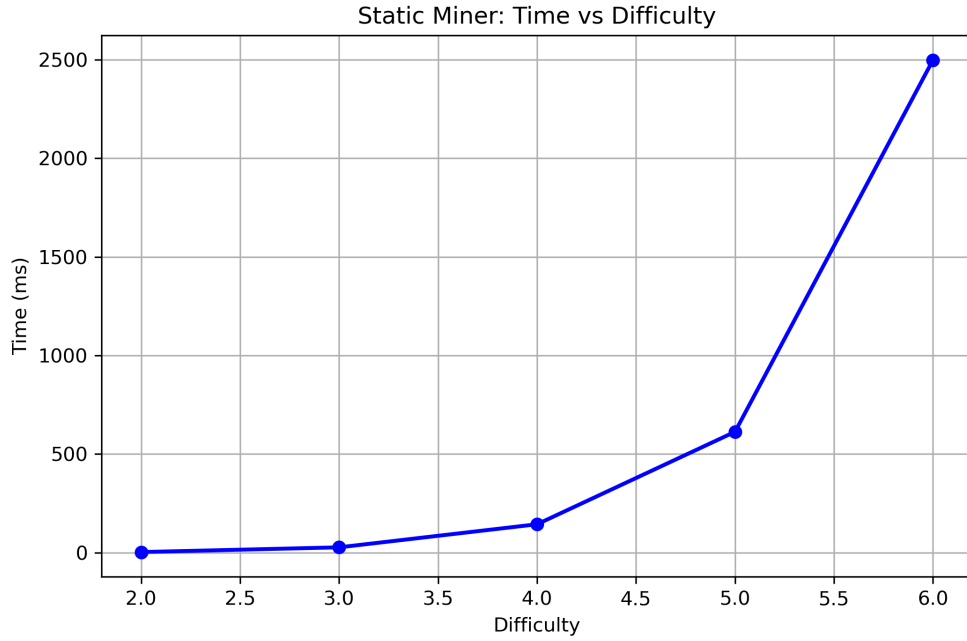


Figure 2: Static Miner: Time vs Difficulty

Observations

- Time decreases as thread count increases.
- Time increases sharply with difficulty.

4.2 Dynamic Work-Stealing Miner Results

4.2.1 Threads vs Time

Threads	Time (ms)
2	1489
4	1347
8	1319
12	1335
16	1116

Table 3: Dynamic Miner: Time vs Threads (difficulty: 5, total-nonces: 50000000, chunk-size: 50000)

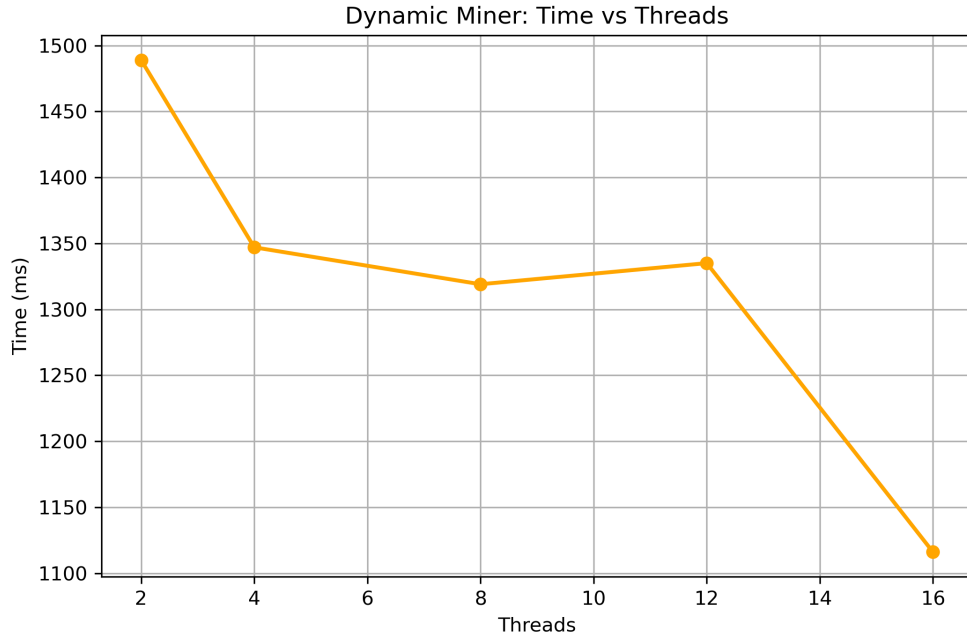


Figure 3: Dynamic Miner: Time vs Threads

4.2.2 Difficulty vs Time

Difficulty	Time (ms)
2	6
3	24
4	145
5	1239
6	8394

Table 4: Dynamic Miner: Time vs Difficulty (threads fixed, Threads:6 total-nonces: 50000000 chunk-size: 50000)

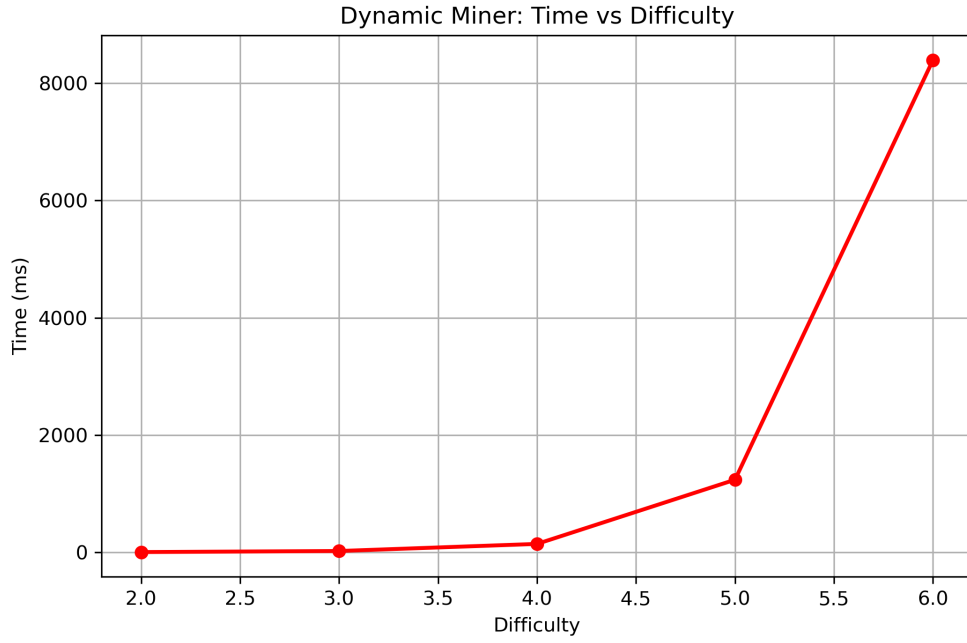


Figure 4: Dynamic Miner: Time vs Difficulty

Observations

- Dynamic miner benefits from load balancing.
- Difficulty strongly affects runtime.

4.3 MPI Distributed Miner Results

Difficulty	Time (s)
1	0.33
2	0.32
3	0.32
4	0.45
5	7.34
6	12.60

Table 5: MPI Miner on 3 VMs: Time vs Difficulty (3 processes, 1 per VM)

Difficulty	Time (s)
1	0.24
2	0.23
3	0.24
4	0.39
5	3.80
6	6.30

Table 6: MPI Miner on Local Machine: Time vs Difficulty (3 processes)

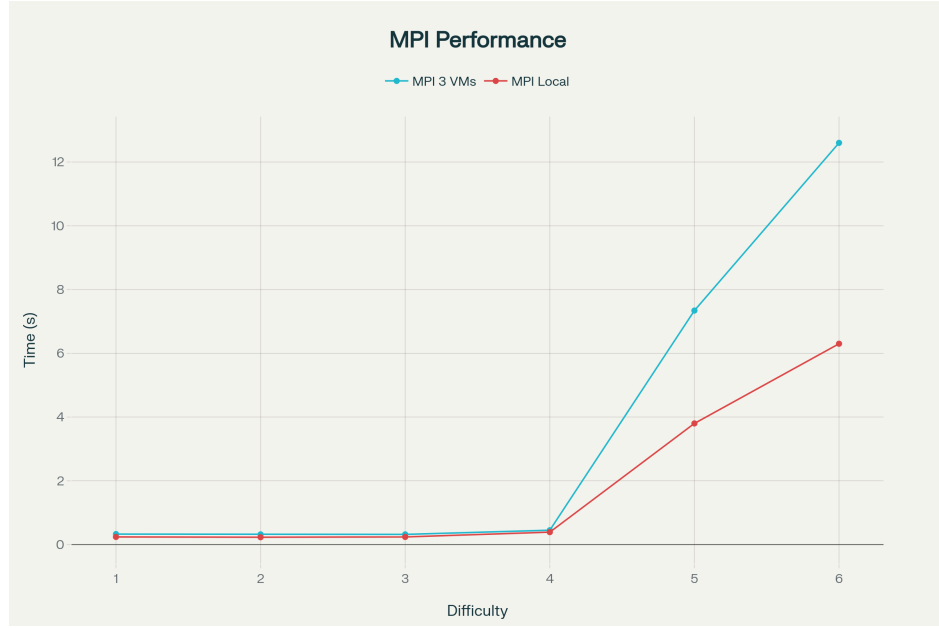


Figure 5: MPI Miner: Time vs Difficulty Comparison (Distributed vs Local)

4.3.1 Processes vs Time (Difficulty 5)

Processes	Time (s)
2	3.55 (unstable ranges in either 0.4-0.5 or in 7.2-7.6)
4	2.80 (unstable either in 0.6-0.8 or 4.3 - 4.8)
8	0.58 (unstable similar to 4)
12	1.11 (stable)

Table 7: MPI Miner on Local Machine: Time vs Processes (Difficulty 5)

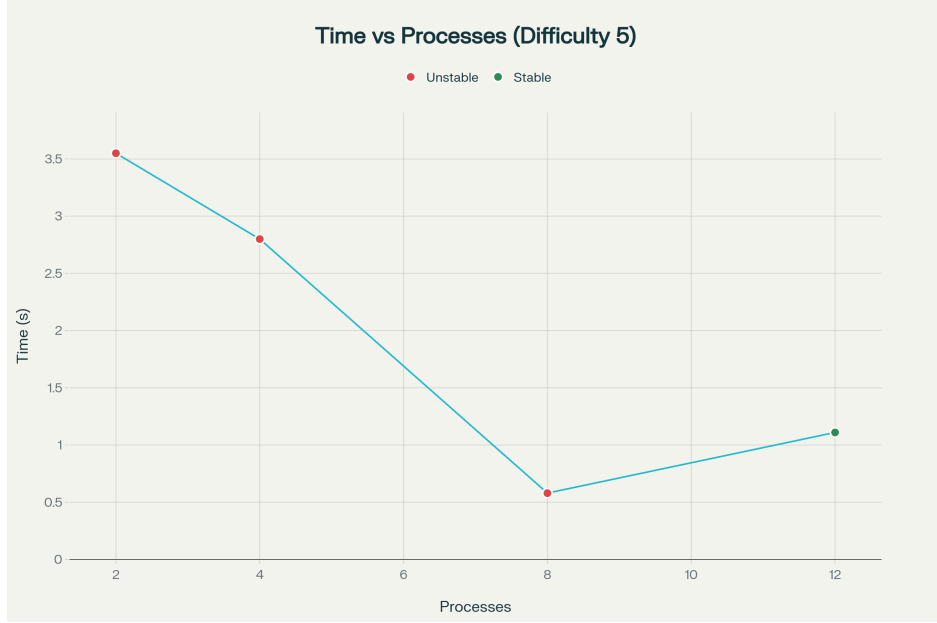


Figure 6: MPI Miner: Time vs Number of Processes (Difficulty 5)

Observations

- **VM vs Local Performance:** The local machine consistently outperforms the 3-VM distributed setup across all difficulties. At difficulty 5, the local machine completes mining in 3.8s compared to 7.34s on VMs, demonstrating nearly 2x speedup due to reduced network latency and communication overhead in the distributed system.
- **Exponential Growth with Difficulty:** Mining time increases exponentially with difficulty level. For difficulties 1-4, execution completes in under 0.5s, while difficulty 5 requires 3.8-7.34s and difficulty 6 takes 12.6s on VMs, showing the computational impact of additional leading zero requirements.
- **Optimal Process Count:** With fixed difficulty 5, performance improves from 2 to 8 processes (3.55s to 0.58s), achieving a 6x speedup. However, scaling to 12 processes shows diminishing returns with time increasing to 1.11s, likely due to increased coordination overhead and work stealing inefficiency when the search space is divided too finely.
- **Communication Overhead:** The distributed VM setup exhibits longer execution times despite using the same process count as local runs, highlighting the cost of MPI inter-node communication compared to intra-node communication.

5 Overall Comparison

Method	Time (ms)	Notes
Static Partition	141 ms	Fastest raw hashing; no runtime balancing
Dynamic Stealing	1116 ms	Balanced workload; overhead of stealing
MPI Distributed (Local, 3 procs)	3800 ms	Inter-process overhead; scalable across machines

Table 8: Comparison of All Three Mining Approaches at Difficulty 5

Comparison Summary

The three mining methods show fundamentally different performance characteristics:

- **Static Miner:** Achieves the lowest execution time (141 ms) because threads operate independently with no synchronization once partitions are assigned. It benefits from minimal overhead but suffers from potential imbalance.
- **Dynamic Miner:** Takes longer (1116 ms) because stealing, chunk distribution, and lock-handling add cost. However, runtime is far more stable across different executions and difficulty levels because threads rarely idle.
- **MPI Miner:** Significantly slower on a single machine (3800 ms) due to messaging, checkpointing, leader heartbeats, and work reassignments. It is the only method that scales horizontally across physical machines and handles failures using Raft.

Overall, the static miner excels in raw speed, the dynamic miner excels in CPU utilization, and the MPI miner excels in distributed resilience and scalability.

6 Final Observations and Insights

Across all three mining implementations—static partitioning, dynamic work stealing, and distributed MPI-based mining—we observe clear differences in performance, scalability, and system behavior.

1. Static Miner: Fast but Imbalanced

The static miner divides the nonce range into equal partitions without any runtime redistribution. This simplicity results in:

- **Near-linear speedup** as threads increase (2 $\rightarrow$ 16 threads shows a strong drop in runtime).
- **Severe imbalance at higher difficulties:** When the winning nonce lies disproportionately early or late, some threads finish early and remain idle.
- **Best single-machine raw speed** at high thread counts because no synchronization or work-sharing overhead exists.

This method performs well when work is uniformly hard, but poorly when the winning nonce appears early in a heavily loaded thread.

2. Dynamic Miner: Load Balancing Improves Stability

The dynamic miner addresses static imbalance using chunk-based distribution and work stealing. From both code traces and experimental data:

- **All threads remain productive** due to fine-grained chunks and stealing.
- **Stable runtime:** Even when the solution appears early, work redistributes quickly.
- **Higher overhead at low difficulties** where chunk exchange and locking cost more than actual hashing.
- **Better scaling for difficult problems:** performance becomes smoother and predictable once difficulty (and thus runtime) increases.

Dynamic stealing therefore trades some raw speed for fairness, consistency, and adaptability.

3. MPI Distributed Miner: True Distributed Scalability, but Higher Overhead

The MPI miner extends parallelism across machines using message passing, Raft-based leader election, and fault-tolerant work management. From the measured results:

- **Local MPI processes outperform multi-VM clusters:** Network latency significantly slows range assignment, heartbeats, and checkpoints. Local 3-process runs nearly halve the time compared to 3 VMs.
- **Difficulty dominates time:** Difficulties 1–4 complete almost instantly (<0.5 s), while difficulty 6 takes 6–12 s depending on cluster topology. This matches expected exponential SHA-256 search growth.
- **Non-monotonic scaling with process count:** Increasing processes from 2→8 reduces runtime sharply (down to 0.58 s), but adding more (8→12 processes) introduces:
 - More heartbeats and election traffic,
 - More work-range assignments,
 - Higher synchronization costs.

Thus, speedup saturates and may even regress.

- **Robustness via Raft:** The miner continues operating correctly even if a leader stalls, work is lost, or a worker becomes slow. Work ranges can be reassigned, and elections restart reliably.

The distributed miner provides reliability and true multi-machine parallelism at the cost of increased communication overhead.

4. Combined System-Level Insight

- **Static = Maximum raw throughput, minimum overhead.** Best for controlled environments with predictable uniform workloads.
- **Dynamic = Balanced and efficient CPU usage.** Best for minimizing idle time when workloads are uneven or when solutions appear unpredictably.
- **MPI = Horizontal scaling + fault tolerance.** Best when distributing computation across machines, at the cost of communication overhead and leadership coordination.

7 Conclusion

Each miner excels in different conditions:

- Static miner is fastest in ideal single-machine cases.
- Dynamic miner is most consistent when nonce discovery is unpredictable.
- MPI miner is the only approach that scales beyond one machine and handles real distributed-systems challenges such as failures and load spikes.

The experimental data, code behavior, and observed scaling trends collectively show that distributed mining introduces complexity in exchange for robustness and flexibility, whereas static and dynamic CPU miners optimize pure hashing throughput.